1.Write a program for designing star.

Code for designing colorful star:

```
#include <GL/gl.h>
#include <GL/glut.h>
void display(void)
{
/* clear all pixels */
glClear (GL_COLOR_BUFFER_BIT);
/* draw white polygon (rectangle) with corners at
* (0.25, 0.25, 0.0) and (0.75, 0.75, 0.0)
*/


///1st body
glColor3f (1.0, 1.0, 1.0);
glBegin(GL_POLYGON);
glVertex3f (0.53, 0.42, 0.0);
glVertex3f (0.56, 0.34, 0.0);
glVertex3f (0.59, 0.42, 0.0);
glVertex3f (0.67, 0.45, 0.0);
glVertex3f (0.59, 0.48, 0.0);
glVertex3f (0.56, 0.56, 0.0);
glVertex3f (0.53, 0.48, 0.0);
glVertex3f (0.45, 0.45, 0.0);
glEnd();
///2nd body
glColor3f (1.0, 0.0, 0.0);
glBegin(GL_POLYGON);
glVertex3f (0.31, 0.42, 0.0);
glVertex3f (0.34, 0.34, 0.0);
glVertex3f (0.37, 0.42, 0.0);
glVertex3f (0.45, 0.45, 0.0);
glVertex3f (0.37, 0.48, 0.0);
glVertex3f (0.34, 0.56, 0.0);
glVertex3f (0.31, 0.48, 0.0);
glVertex3f (0.23, 0.45, 0.0);
glEnd();
///2nd body
glColor3f (0.0, 1.0, 0.0);
glBegin(GL_POLYGON);
glVertex3f (0.42, 0.53, 0.0);
glVertex3f (0.45, 0.45, 0.0);
glVertex3f (0.48, 0.53, 0.0);
glVertex3f (0.56, 0.56, 0.0);
glVertex3f (0.48, 0.59, 0.0);
```

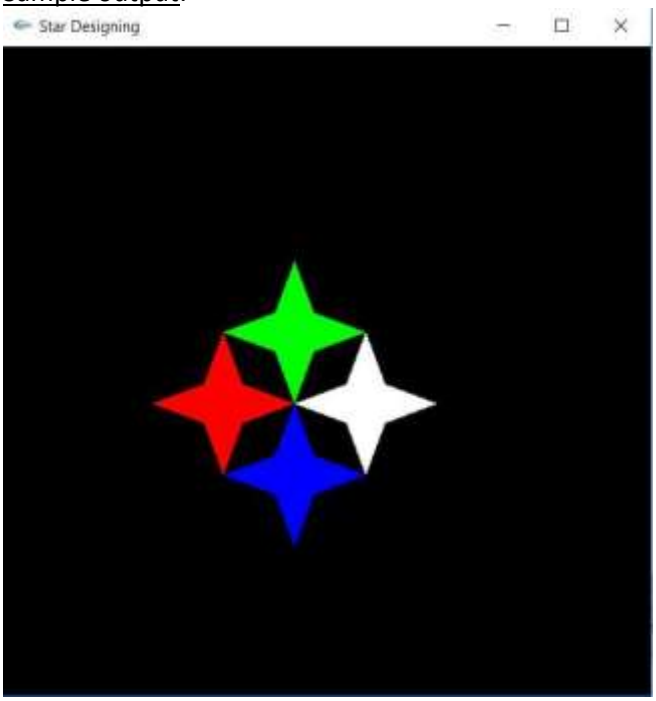
```
glVertex3f (0.45, 0.67, 0.0);
glVertex3f (0.42, 0.59, 0.0);
glVertex3f (0.34, 0.56, 0.0);
glEnd();
///2nd body
glColor3f (0.0, 0.0, 1.0);
glBegin(GL_POLYGON);
glVertex3f (0.42, 0.31, 0.0);
glVertex3f (0.45, 0.23, 0.0);
glVertex3f (0.48, 0.31, 0.0);
glVertex3f (0.56, 0.34, 0.0);
glVertex3f (0.48, 0.37, 0.0);
glVertex3f (0.45, 0.45, 0.0);
glVertex3f (0.42, 0.37, 0.0);
glVertex3f (0.34, 0.34, 0.0);
glEnd();
```

```
/* don't wait!
* start processing buffered OpenGL routines
*/
glFlush ();
}
void init (void)
{
/* select clearing (background) color */
glClearColor (0.0, 0.0, 0.0, 0.0);
/* initialize viewing values */
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glOrtho(0.0, 1.0, 0.0, 1.0, -1.0, 1.0);
}
/*
* Declare initial window size, position, and display mode
* (single buffer and RGBA). Open window with "hello"
* in its title bar. Call initialization routines.
* Register callback function to display graphics.
* Enter main loop and process events.
*/
int main(int argc, char** argv)
{
glutInit(&argc, argv);
glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
glutInitWindowSize (500, 500);
```

```
glutInitWindowPosition (100, 100);
glutCreateWindow ("Star Designing");
init ();
glutDisplayFunc(display);
glutMainLoop();
return 0; /* ISO C requires main to return int. */
}
```

Sample input:
No sample input is needed.

Sample output:



2.Drawing a Multicolor Triangle.

Source Code for **Drawing a Multicolor Triangle:**

```
#include <GL/glut.h>

void displayMe(void)
{
    glClear(GL_COLOR_BUFFER_BIT);

    glBegin(GL_POLYGON);
```

```
            glColor3f(1.0,0.0,2.0);
        glVertex3f(0.10, 0.25, 0.0);

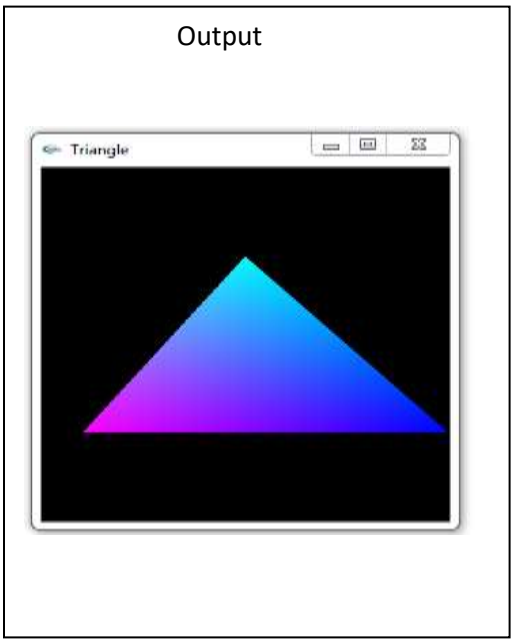
                glColor3f(0.0,1.0,2.0);
            glVertex3f(0.5, 0.75, 0.0);

                glColor3f(0.0,0.0,2.0);
         glVertex3f(1.0, 0.25, 0.0);

    glEnd();
    glFlush();
}

void init(void)
       {
         glClearColor(0.0,0.0,0.0,0.0);
         glMatrixMode(GL_PROJECTION);
         glLoadIdentity();
         glOrtho(0.0,1.0,0.0,1.0,-1.0,1.0);
  }

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(300, 300);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("Triangle");
       init();
    glutDisplayFunc(displayMe);
    glutMainLoop();
    return 0;
}
```



3. Drawing a Multicolor Square using Opengl.

Source Code For Drawing a Multicolor Square using Opengl:

```
#include <cstdlib>
#include <GL/glut.h>
#include <GL/glut.h>

void display()
{
    // clear the draw buffer .
    glClear(GL_COLOR_BUFFER_BIT);   // Erase everything
```

```
    // set the color to use in draw
    glColor3f(0.5, 0.5, 0.0);
    // create a polygon ( define the vertexs)
    glBegin(GL_POLYGON); {
              glColor3f(0.8, 0.0, 0.0);
        glVertex2f(-0.5, -0.5);
              glColor3f(0.0, 0.0, 0.8);

        glVertex2f(-0.5,  0.5);
              glColor3f(0.0, 0.7, 0.0);
        glVertex2f( 0.5,  0.5);
              glColor3f(0.0, 0.0, 0.3);
        glVertex2f( 0.5, -0.5);
    } glEnd();

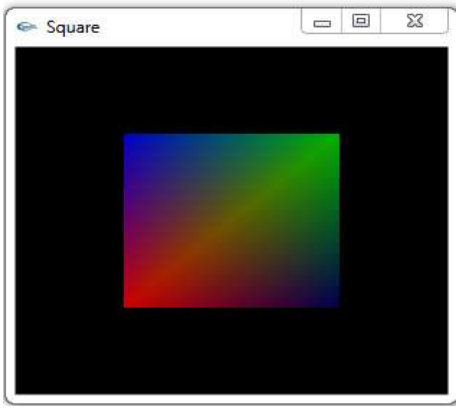
    // flush the drawing to screen .
    glFlush();
}

// Keyboard callback function ( called on keyboard event handling )
void keyboard(unsigned char key, int x, int y)
{
    if (key == 'q' || key == 'Q')
        exit(EXIT_SUCCESS);
}

// Main execution  function
int main(int argc, char *argv[])
{
    glutInit(&argc, argv);      // Initialize GLUT
    glutCreateWindow("OpenGL example");  // Create a window
    glutDisplayFunc(display);   // Register display callback
    glutKeyboardFunc(keyboard); // Register keyboard callback
    glutMainLoop();             // Enter main event loop

    return (EXIT_SUCCESS);
}
```

Output



4. Drawing a Multicolor Polygon using Opengl.

Source Code for Drawing a Multicolor Polygon using Opengl **:**

```
#include <GL/glut.h>

void displayMe(void)
{
    glClear(GL_COLOR_BUFFER_BIT);

        glBegin(GL_POLYGON);
              glColor3f(0.0,4.0,7.0);
             glVertex3f(.50,.75,.52);
```

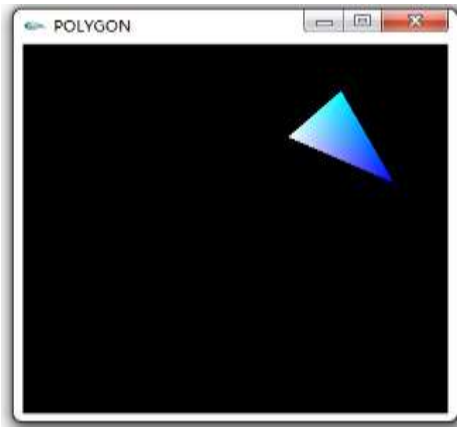
```
            glColor3f(0.0,0.0,7.0);
            glVertex3f(.75,.25,.25);

            glColor3f(0.0,4.0,7.0);
            glVertex3f(.25,.50,.75);

    glEnd();
    glFlush();
}


int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(300, 300);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("POLYGON");
    glutDisplayFunc(displayMe);
    glutMainLoop();
    return 0;
}
```

Output



5. Perform some design using OpenGL

Source Code

```
#include <GL/gl.h>

#include <GL/glut.h>
void display(void)
{
/* clear all pixels */
glClear (GL_COLOR_BUFFER_BIT);
/* draw white polygon (rectangle) with corners at
```

```
* (0.25, 0.25, 0.0) and (0.75, 0.75, 0.0)
*/


///1st body
glColor3f (1.0, 1.0, 1.0);
glBegin(GL_POLYGON);
glVertex3f (0.25, 0.25, 0.0);
glVertex3f (0.75, 0.25, 0.0);
glVertex3f (0.75, 0.75, 0.0);
glVertex3f (0.25, 0.75, 0.0);
glEnd();


///bottom side
glColor3f (1.0, 0.75, 0.75);
glBegin(GL_POLYGON);
glVertex3f (0.20, 0.20, 0.0);
glVertex3f (0.80, 0.20, 0.0);
glVertex3f (0.75, 0.25, 0.0);
glVertex3f (0.25, 0.25, 0.0);
glEnd();
///glColor3f (0.5, 0.5, 0.5);
glColor3f (0.0, 1.00, 0.75);
glBegin(GL_POLYGON);
glVertex3f (0.20, 0.15, 0.0);
glVertex3f (0.80, 0.15, 0.0);
glVertex3f (0.80, 0.20, 0.0);
glVertex3f (0.20, 0.20, 0.0);
glEnd();

///Stairs

glColor3f (0.5, 0.5, 0.5);
glBegin(GL_POLYGON);
glVertex3f (0.0, 0.20, 0.0);
glVertex3f (0.20, 0.20, 0.0);
glVertex3f (0.25, 0.25, 0.0);
glVertex3f (0.0, 0.25, 0.0);
glEnd();
glColor3f (0.5, 0.5, 0.5);
glBegin(GL_POLYGON);
```

```
glVertex3f (0.0, 0.15, 0.0);
glVertex3f (0.20, 0.15, 0.0);
glVertex3f (0.20, 0.20, 0.0);
glVertex3f (0.0, 0.20, 0.0);
glEnd();
glColor3f (0.5, 0.5, 0.5);
glBegin(GL_POLYGON);
glVertex3f (0.80, 0.15, 0.0);
glVertex3f (1.0, 0.15, 0.0);
glVertex3f (1.0, 0.20, 0.0);
glVertex3f (0.80, 0.20, 0.0);
glEnd();
glColor3f (0.5, 0.5, 0.5);
glBegin(GL_POLYGON);
glVertex3f (0.80, 0.20, 0.0);
glVertex3f (1.0, 0.20, 0.0);
glVertex3f (1.0, 0.25, 0.0);
glVertex3f (0.75, 0.25, 0.0);
glEnd();

///top side
glColor3f (0.0, 1.0, 0.0);
glBegin(GL_TRIANGLES);
glVertex3f (0.25, 0.75, 0.0);
glVertex3f (0.75, 0.75, 0.0);
glVertex3f (0.50, 0.99, 0.0);
glEnd();


///door
glColor3f (0.0, 0.0, 1.0);
glBegin(GL_POLYGON);
glVertex3f (0.45, 0.25, 0.0);
glVertex3f (0.55, 0.25, 0.0);
glVertex3f (0.55, 0.60, 0.0);
glVertex3f (0.45, 0.60, 0.0);
glEnd();


///window
glColor3f (1.0, 0.0, 0.0);
glBegin(GL_POLYGON);
```

```
glVertex3f (0.30, 0.45, 0.0);
glVertex3f (0.40, 0.45, 0.0);
glVertex3f (0.40, 0.55, 0.0);
glVertex3f (0.30, 0.55, 0.0);
glEnd();
glColor3f (1.0, 0.0, 0.0);
glBegin(GL_POLYGON);
glVertex3f (0.60, 0.45, 0.0);
glVertex3f (0.70, 0.45, 0.0);
glVertex3f (0.70, 0.55, 0.0);
glVertex3f (0.60, 0.55, 0.0);
glEnd();

/* don't wait!
* start processing buffered OpenGL routines
*/
glFlush ();
}
void init (void)
{
/* select clearing (background) color */
glClearColor (0.0, 0.0, 0.0, 0.0);
/* initialize viewing values */
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glOrtho(0.0, 1.0, 0.0, 1.0, -1.0, 1.0);
}
/*
* Declare initial window size, position, and display mode
* (single buffer and RGBA). Open window with "hello"
* in its title bar. Call initialization routines.
* Register callback function to display graphics.
* Enter main loop and process events.
*/
int main(int argc, char** argv)
{
glutInit(&argc, argv);
glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
glutInitWindowSize (500, 500);
glutInitWindowPosition (100, 100);
glutCreateWindow ("hello");
init ();
```

```
glutDisplayFunc(display);
glutMainLoop();
return 0; /* ISO C requires main to return int. */
}
```

Output:

